El gran cambio de paradigma en el Backend.

Dejando atrás el modelo bloqueante para dominar el motor asíncrono de Node.js.

Gemini Notebook



El problema del Modelo Tradicional

En entornos clásicos (PHP, Java), el paradigma era "Una petición = Un hilo dedicado".

Espera bloqueante: El hilo se congela esperando respuestas lentas de bases de datos o red.

Costo de escala: Alto consumo de memoria RAM y cambios de contexto abusivos por parte del procesador.

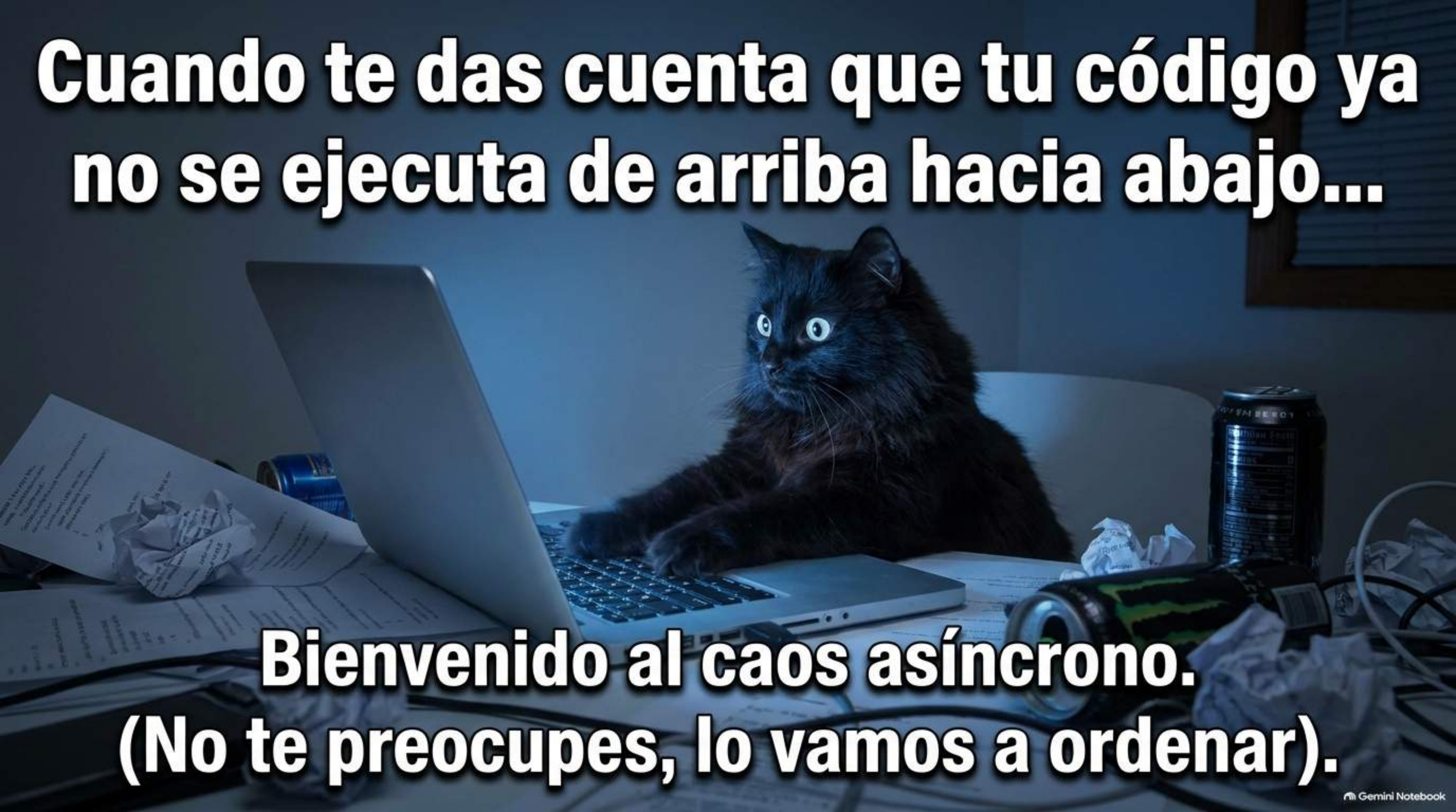
El Riesgo: Agotamiento total de hilos bajo alta concurrencia. El servidor deja de responder.

Gemini Notebook

El Choque de Mundos: Hilos vs. Eventos

Modelo Tradicional (Multi-hilo)	Modelo Node.js (Event-driven)
Un hilo pesado y dedicado por cada petición.	Un hilo principal ligero + Pool de workers nativos.
Hilo bloqueado de forma ociosa esperando E/S (disco, red).	Espera delegada al SO; el hilo se libera instantáneamente.
Alto costo de memoria bajo picos de tráfico extremo.	Bajo costo de memoria, ideal para miles de conexiones concurrentes.
Modelo mental: Ejecución síncrona y lineal.	Modelo mental: Las peticiones son eventos; los callbacks manejan los resultados.

Gemini Notebook


Cuando te das cuenta que tu código ya no se ejecuta de arriba hacia abajo...
Bienvenido al caos asíncrono.
(No te preocupes, lo vamos a ordenar).
Gemini Notebook

El Motor Bajo el Capó: Los 3 Pilares

V8 Engine

El motor de Google. Es un compilador JIT escrito en C++. Traduce tu código fuente JavaScript a código máquina de altísimo rendimiento en tiempo de ejecución.

Node APIs

El puente de comunicación. Son las interfaces que exponen las funcionalidades internas del sistema (como fs, http, crypto) hacia tu entorno JavaScript.

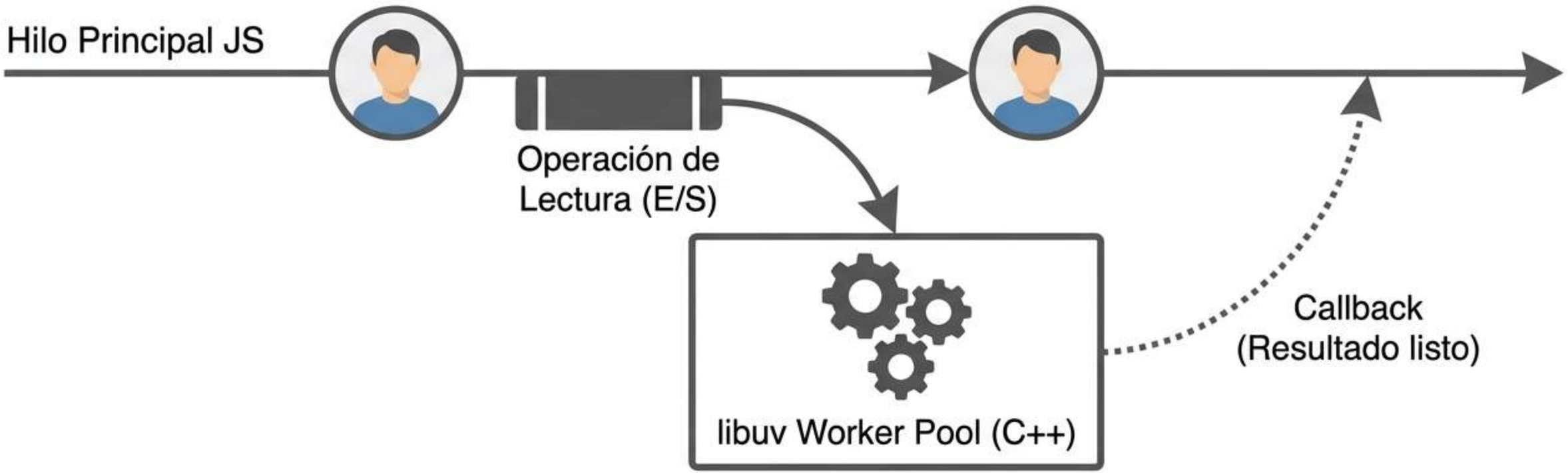
libuv

El cerebro asíncrono. Una biblioteca nativa en C++ que gestiona el Event Loop, las colas del sistema operativo y el Pool de Hilos (Workers) para tareas pesadas.

Gemini Notebook

El mito del Single-Thread

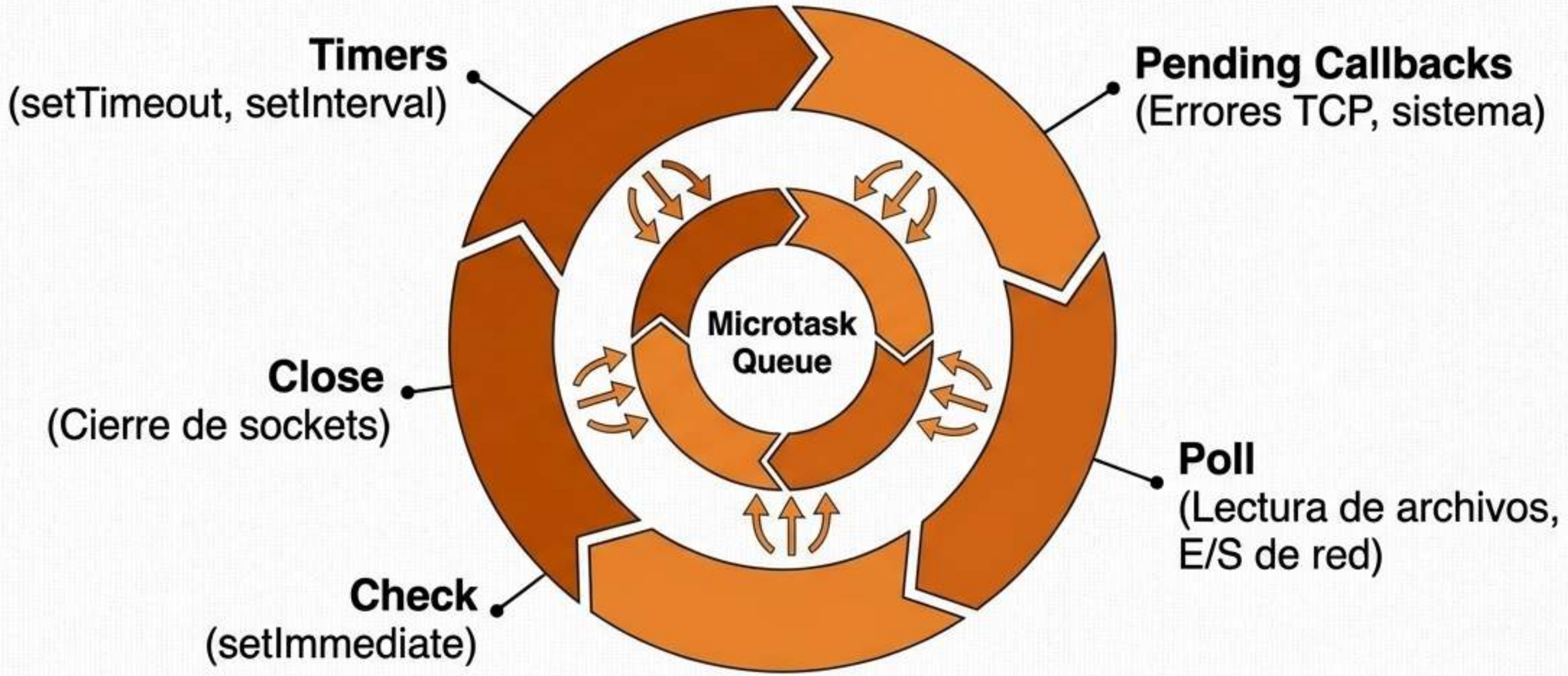
NUNCA asumas que Node es single-threaded en sentido absoluto. JavaScript corre en un hilo, el entorno no.



1. Llega petición asíncrona -> **2. Delega** la carga pesada al OS -> **3.** Hilo principal liberado **de inmediato** -> **4. Callback** ejecutado al terminar.

Gemini Notebook

Anatomía del Event Loop



process.nextTick() y las Promesas resueltas tienen prioridad absoluta y detienen el avance del loop hasta vaciarse.

Gemini Notebook

La Regla de Oro del Backend:

NUNCA BLOQUEES EL EVENT LOOP.

El Trade-off Fundamental

Node.js es imbatible para tareas I/O-bound (APIs, proxies, bases de datos). Es deficiente para tareas CPU-bound largas (procesamiento de imágenes, minería de datos) sin usar arquitecturas adicionales.

Las Reglas del Juego:

- Mantén los callbacks y middlewares extremadamente cortos.
- Jamás utilices métodos de sistema síncronos (ej. readFileSync) en rutas activas.
- Diseña pensando en backpressure: controla el volumen de datos en memoria.

Gemini Notebook

Fronteras del Entorno: Frontend vs. Backend

	Navegador	Node.js
Objeto Global	**window**	**global**
Interfaz y DOM	❌ document, alert	❌ No existe interfaz gráfica
Sistema de Archivos	❌ Sandbox de seguridad	✅ Acceso total nativo (módulo fs)
Sistema Operativo	❌ Aislado	✅ Control profundo (objeto process)

Regla Crítica: El ámbito superior de un módulo en Node es local, no global. Nunca uses variables globales como estado compartido; arruinarás la concurrencia de los usuarios.

Gemini Notebook

Momentos de la Clase: Ejemplo en Vivo

Configuración Inicial y ESM

File Edit Selection View Go Run Terminal Help package.json - Visual Studio Code

{} package.json

{} package.json > {} package.json > ...

```
1  {
2    "name": "coder-backend",
3    "type": "module",
4    "scripts": {
5      "dev": "node --watch server.js"
6    }
7  }
```

JS server.js

JS server.js > ...

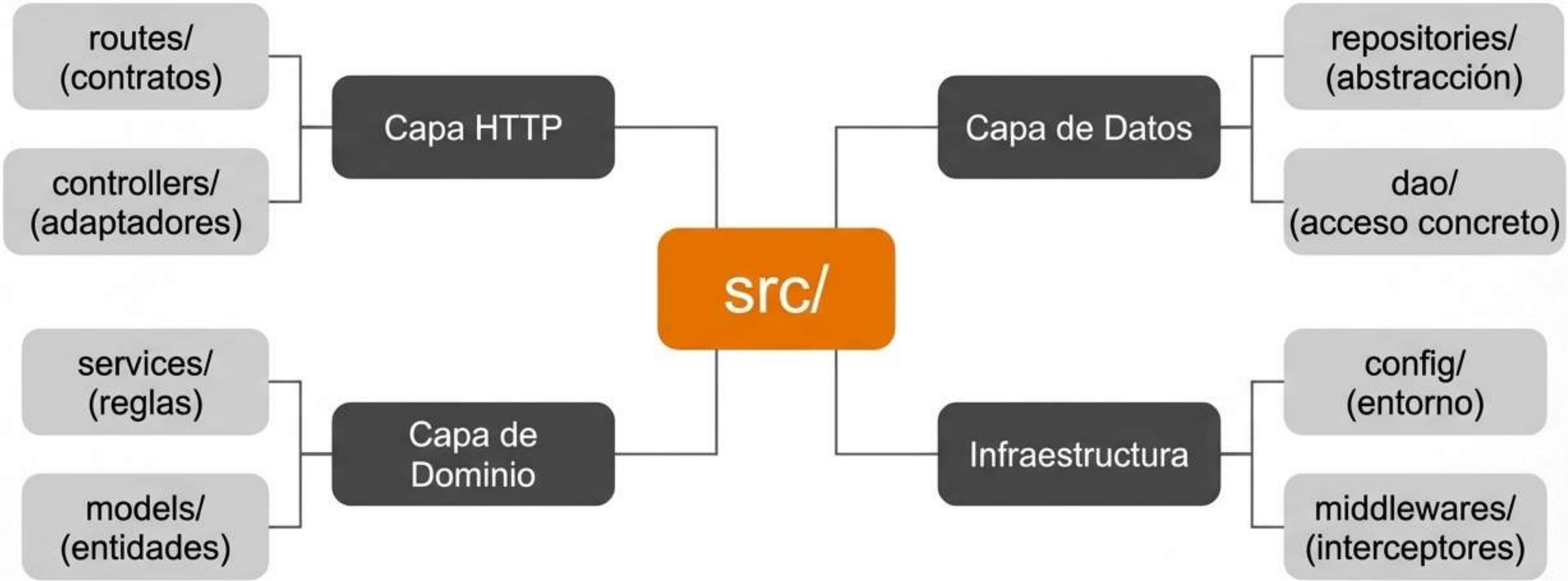
```
1  import { app } from './app.js';
2  import config from './config/env.js';
3
4  app.listen(config.port, () => {
5    console.log('Servidor asíncrono online');
6  });
```

En ESM, incluir la extensión **.js** en las importaciones locales es absolutamente obligatorio.



Arquitectura Profesional: Separación de Responsabilidades

La estructura de carpetas debe expresar el dominio, no los tipos de archivos.



Gemini Notebook

El primer límite estricto: app.js vs server.js

app.js

El Entorno de la Aplicación

- Instancia la aplicación Express.
- Configura todos los Middlewares globales.
- Inyecta el enrutador principal (Routes).
- Exporta la app (NO escucha puertos).

Ideal para realizar testing aislado sin levantar servidores fantasma.

server.js

La Capa de Red

- Importa la 'app' y la configuración.
- Establece la conexión a la Base de Datos.
- Maneja procesos del sistema (Workers).
- Ejecuta app.listen(PORT).

Responsabilidad única y exclusiva de levantar la infraestructura.

Gemini Notebook

Gestión Segura y Metodología 12-Factor App



.env

Contiene valores, contraseñas y secretos locales reales. Regla de oro: NUNCA se versiona, debe ir directo al archivo .gitignore.



.env.example

Plantilla con claves vacías para que el equipo de desarrollo sepa qué variables existen. Regla: SÍ se versiona en el repositorio.



Fail-Fast Validation

Dentro de src/config/, un validador verifica que todas las variables existan antes de arrancar. Si falta una credencial crítica, se apaga el proceso inmediatamente.

Gemini Notebook

Momentos de la Clase: Actividad colaborativa

Breakout Room: El cuello de botella

El Escenario: Tu servidor Node.js funciona perfecto. De repente, un usuario sube un archivo de 5GB y se dispara una función de compresión síncrona en el hilo principal. Inmediatamente, los otros 5,000 usuarios conectados conectados experimentan una caída total del servicio.

1. ¿Qué pilar exacto de Node.js acaba de ser vulnerado?
2. ¿En qué fase del Event Loop quedó congelado el servidor?
3. ¿Cómo reestructurarías este proceso para no bloquear al resto de los usuarios?





Síntesis Arquitectónica

01. El Paradigma

La concurrencia masiva se logra mediante eventos y delegación al sistema operativo. Protege tu Event Loop por sobre todas las cosas.

02. La Estructura

Las carpetas deben gritar sus responsabilidades. Desacopla la red HTTP, la lógica de negocio y la persistencia de datos.

03. El Entorno

Trata el sistema de archivos, los procesos y los secretos como fronteras externas e inyectables (12-Factor App).

El código se ejecuta de otra forma. Ahora tú diseñas de otra forma.

Gemini Notebook